

Finite State Machines (Part 2)

Lecture 05

Josh Brake

Harvey Mudd College

Outline

- Synchronizers, metastability, and the MTBF equation
- Three kinds of state machine: canonical, non-canonical, communicating
- Worked example: a square wave with an adjustable duty cycle
- Build 1: switch debouncer (together)
- Build 2: keypad scanner, Lab 3 prep (in groups)
- A machine that breaks the pattern

Learning Objectives

By the end of this lecture you should be able to...

- Explain what metastability is, and use the MTBF equation to justify how many synchronizer flops a design needs.
- Tell the three structural flavors of state machine apart, and say why you would write one rather than another.
- Decompose a timing problem into a small control FSM plus a counter, and design the signals between them.
- Run the six-step FSM design process on an unfamiliar problem without being walked through it.
- Write a Verilog testbench for a design whose inputs depend on its own outputs.

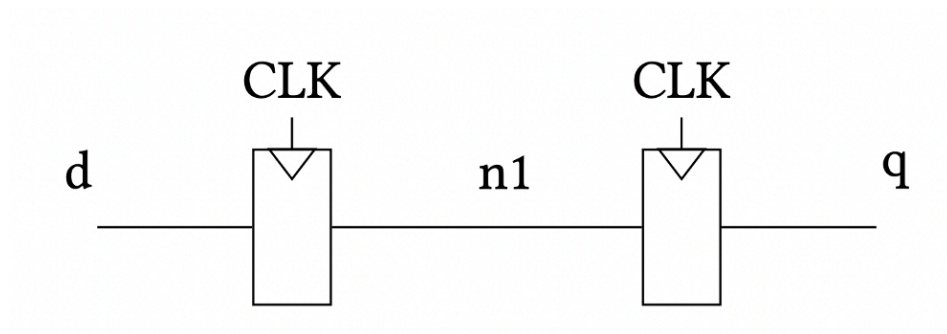
Synchronizers and Metastability

The World Does Not Care About Your Clock

Buttons, keypads, sensors, and other boards change whenever they like. Sooner or later one of them changes _____ of a flop.

When that happens the flop does not output a 0 or a 1. It sits near the switching threshold in a **metastable** state and stays there for an unbounded time.

The Two-Flop Synchronizer



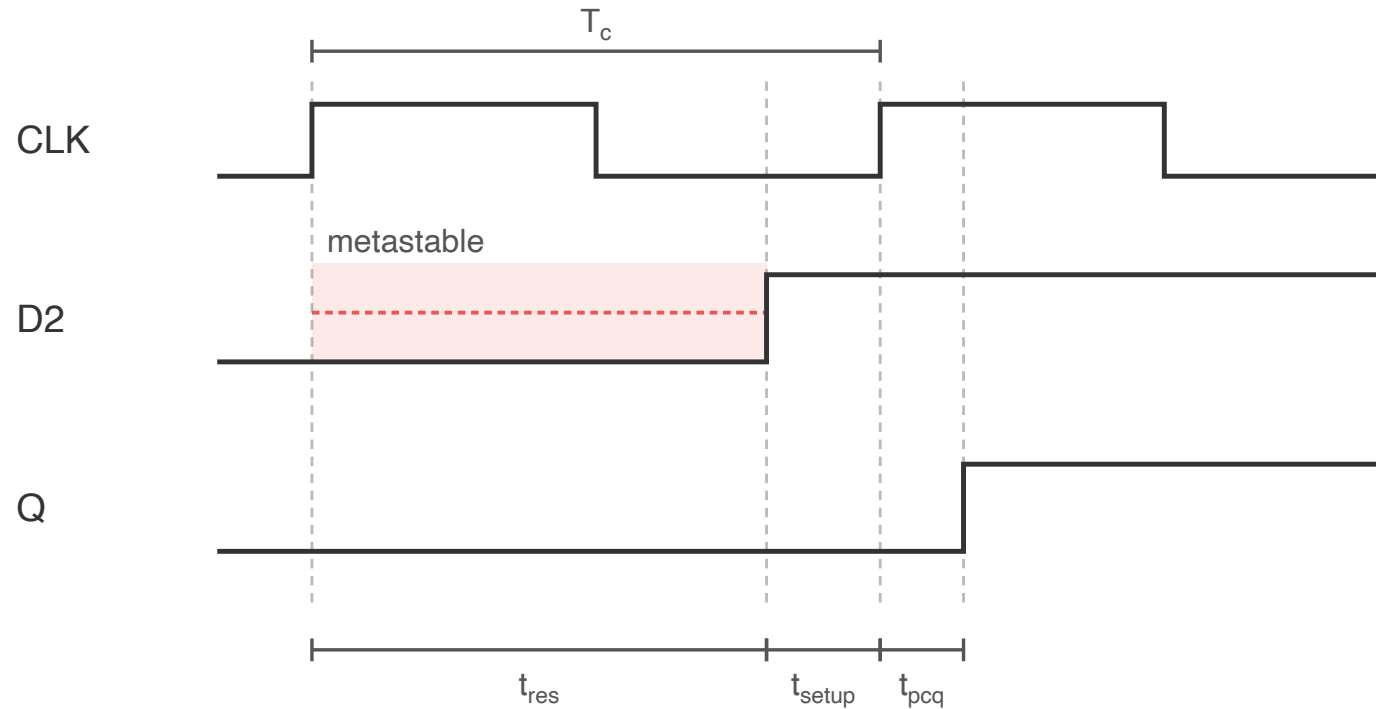
```
1 always_ff @(posedge clk) begin
2     n1 <= d;      // Harris calls this
3     q  <= n1;
4 end
```

Two flops, **F1** and **F2**, with the internal node **D2** between them.

The trick is not that **F1** stops going metastable — it still does. The trick is that **D2** gets _____ to settle before **F2** looks at it.

Q is then clean, at the cost of _____.

When Does a Synchronizer Fail?



Synchronizer fails if $t_{res} > T_c - t_{setup}$

The synchronizer **fails** if **Q** itself comes out metastable. That happens when **D2** has not resolved by the time it has to set up at **F2**:

$$t_{res} > T_c - t_{setup}$$

From Probability to MTBF

Start from how long metastability lasts: after waiting t , the flop is still metastable with probability $\frac{T_0}{T_c} e^{-t/\tau}$.

Substitute the time we actually wait, $T_c - t_{setup}$, to get the failure probability **per input change**:

$$P(\text{failure}) = \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$$

If D changes N times a second, failures per second are N times as likely:

$$P(\text{failure})/\text{sec} = N \frac{T_0}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$$

MTBF is just the reciprocal of that:

$$\text{MTBF} = \frac{1}{P(\text{failure})/\text{sec}} = \frac{T_c e^{\frac{T_c - t_{setup}}{\tau}}}{NT_0}$$

Reading the Equation

Term	Meaning
T_c	clock period — how long we let D2 resolve
t_{setup}	setup time of F2, which eats into that budget
τ	resolution time constant; how fast metastability decays
T_0	aperture width, so T_0/T_c is _____ _____ _____
N	times per second that D changes

Putting Numbers In It

Take $\tau = 0.1$ ns, $T_0 = 0.01$ ns, $t_{setup} = 0.5$ ns, and an input that changes $N = 10$ times a second. Then vary only the clock period:

T_c	f_{clk}	$T_c - t_{setup}$	MTBF
2 ns	500 MHz	1.5 ns	
5 ns	200 MHz	4.5 ns	
10 ns	100 MHz	9.5 ns	
20.8 ns	48 MHz	20.3 ns	

The Fix Is Always One More Cycle

At 500 MHz, add a third flop. Now D2 gets $2T_c - t_{setup} = 3.5$ ns instead of 1.5 ns, and the same equation gives:

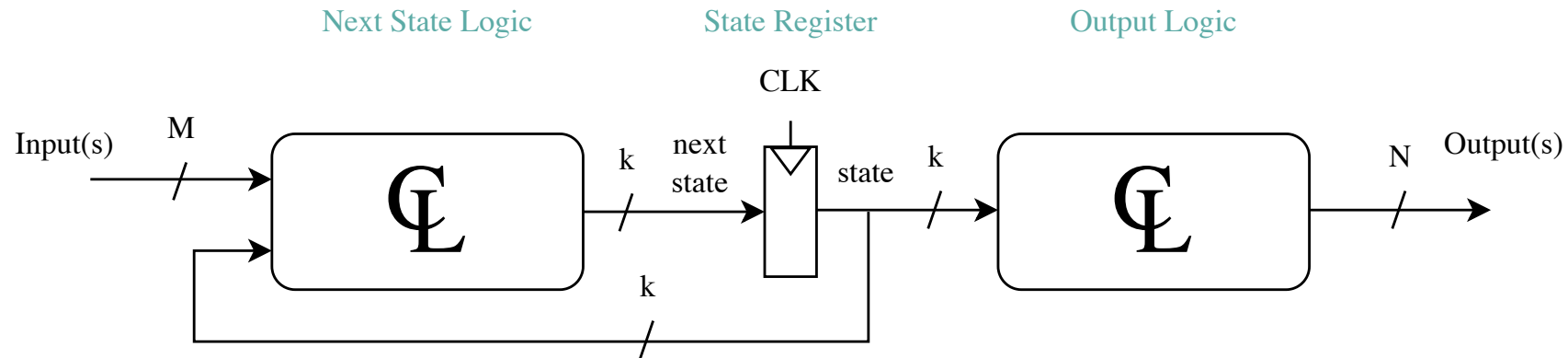
$$\text{MTBF} = \frac{T_c e^{(2T_c - t_{setup})/\tau}}{NT_0}$$

which works out to _____.

Two nanoseconds of extra patience bought a factor of $e^{20} \approx 5 \times 10^8$.

Three Kinds of State Machine

Type 1: The Canonical FSM



Three blocks, always in this order, always written as three separate chunks of Verilog:

1. **Next state logic** — `always_comb` with a `case` on the current state
2. **State register** — `always_ff` with a reset
3. **Output logic** — `always_comb` or `assign`

Type 1: When to Reach for It

Use the canonical form when the state space is _____ — when the interesting content of the design is *which state goes where*, and you want that visible.

What you get: the state diagram and the Verilog are the same object. Anyone can check one against the other.

What it costs: one **case** arm per state. Fine for 3 states, unusable for 300.

Type 2: The Non-Canonical FSM

A counter is a state machine. It has state, it has next state logic, and it has output logic.

```
1 always_ff @(posedge clk, posedge reset)
2     if (reset) count <= 0;
3     else count <= count + 1;
```

But we don't write it as a `case` statement, because a 20-bit counter has _____
_____.

Type 2: Same Hardware, Different Description

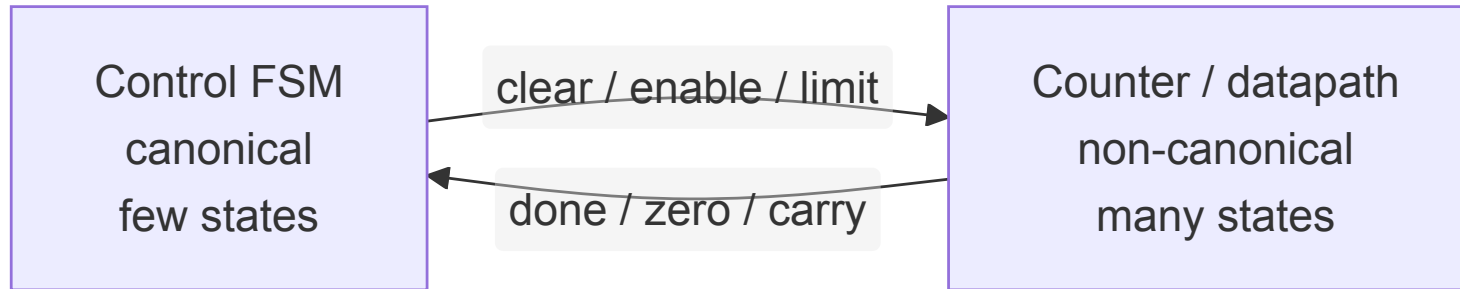
This is not a different kind of circuit. Both types synthesize to the same thing: flip-flops with combinational logic in front of them.

What changes is whether you *enumerate* the transitions or *compute* them.

Non-canonical machines you already have:

- Clock divider (Lab 1–3)
- The rotating **rows** register in the keypad scanner
- Shift registers, LFSRs
- The counter we are about to build

Type 3: Communicating FSMs



Rather than build one machine, build two and let them talk.

- **Down:** control signals — `clear`, `enable`, `load`, a limit value
- **Up:** status signals — `done`, `zero`, `carry`

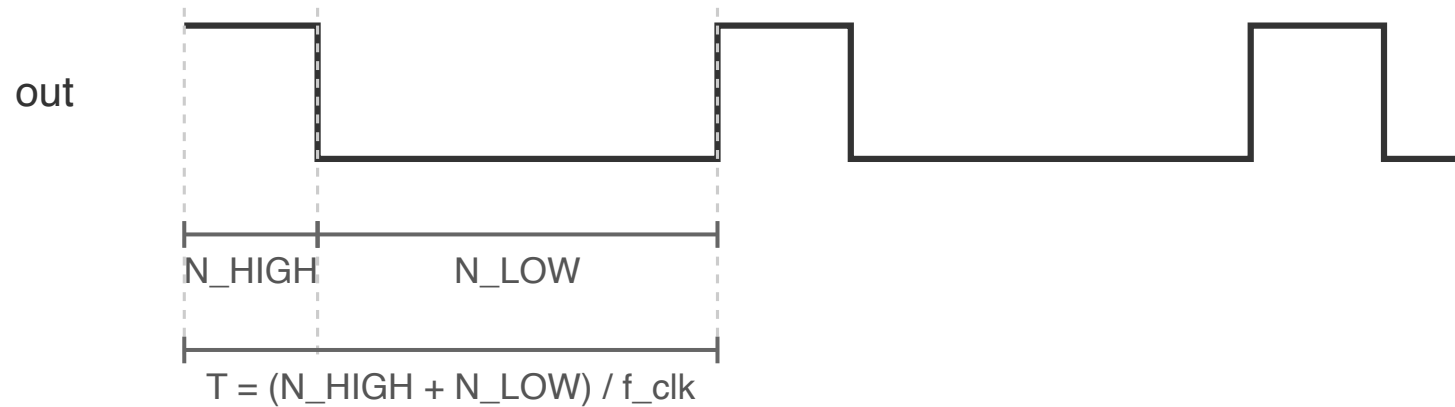
Type 3: Why Bother Splitting?

The machine we are about to build has a 2-state controller and a counter that reaches 36,000. Fuse them and you have _____.

Kept separate, it is a 2-bubble diagram plus one line of arithmetic.

Type 3 in Practice: Variable Duty Cycle

The Specification



Generate a square wave at frequency f_{out} with duty cycle D , from a 48 MHz clock.

$$f_{out} = \frac{f_{clk}}{N_H + N_L} \quad D = \frac{N_H}{N_H + N_L}$$

The catch: the two states have to last **different amounts of time**.

Step 1: Two Machines, Not One

The controller (canonical, Type 1)

Two states, **HIGH** and **LOW**. It owns the output and decides when to flip.

State	out
HIGH	1
LOW	0

The counter (non-canonical, Type 2)

Counts clock cycles. Knows nothing about duty cycle, frequency, or what the output is doing.

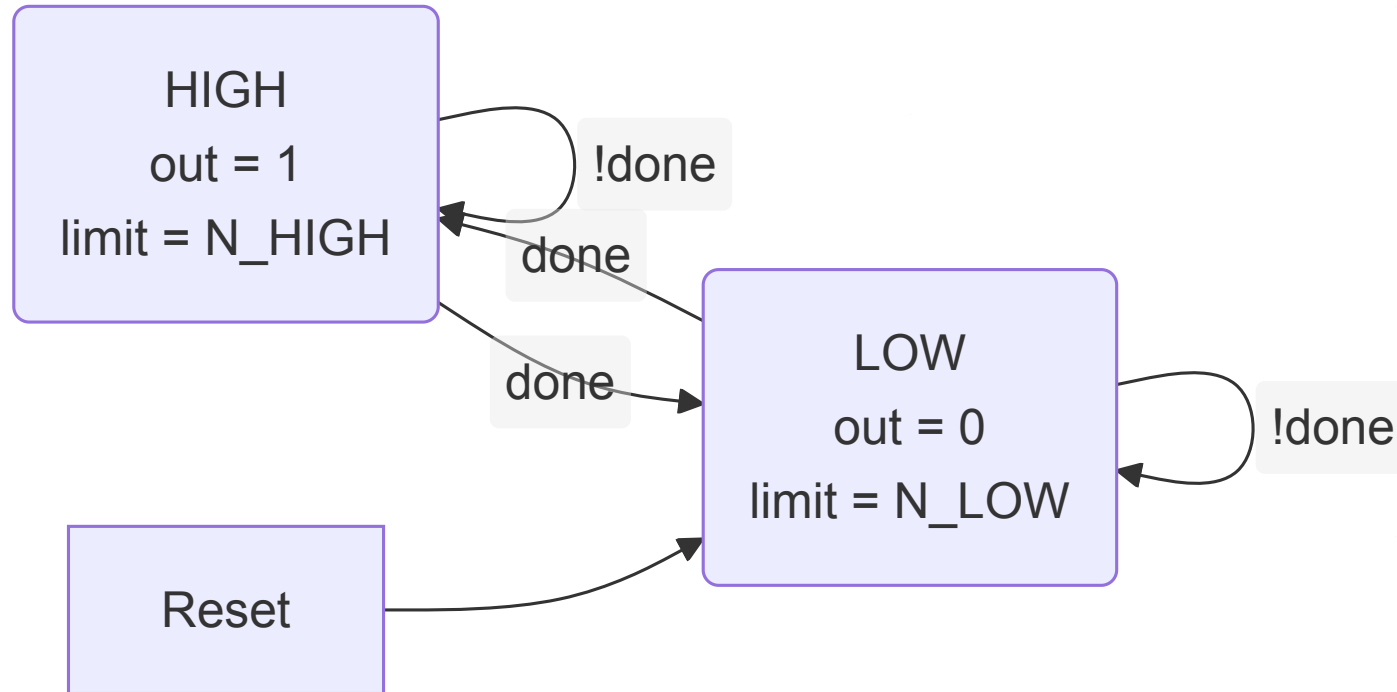
Step 2: Establish How They Communicate

Define the signals that go between the two state machines.

- **Down** — **limit**: how far to count. _____, and that is exactly what buys us an adjustable duty cycle.
- **Up** — **done**: the counter has reached **limit**. The controller uses it as its only transition condition.

```
1 assign limit = (state == HIGH) ? N_HIGH : N_LOW;  
2 assign done  = (count == limit - 1);
```

Step 3: The State Transition Diagram



Two bubbles. All the timing lives in **done**.

Steps 4-7: Declarations and the Counter

```
1 module pwm #(parameter N_HIGH = 12_000, N_LOW = 36_000)
2     (input logic clk, reset, output logic out);
3
4     typedef enum logic {HIGH, LOW} statetype;
5     statetype state, nextstate;
6     logic [15:0] count, limit;
7     logic done;
8
9     // ---- the conversation between the two machines ----
10    assign limit = (state == HIGH) ? N_HIGH : N_LOW;
11    assign done  = (count == limit - 1);
12
13    // ---- machine 2: the counter (non-canonical) ----
14    always_ff @(posedge clk, posedge reset)
15        if (reset)      count <= 0;
16        else if (done) count <= 0;    // the transition clears it
17        else            count <= count + 1;
```

Steps 4-7: The Controller

```
1    // ---- machine 1: the controller (canonical) ----
2    always_ff @(posedge clk, posedge reset)
3        if (reset) state <= LOW;
4        else      state <= nextstate;
5
6    always_comb
7        case (state)
8            HIGH:    nextstate = done ? LOW  : HIGH;
9            LOW:     nextstate = done ? HIGH : LOW;
10           default: nextstate = LOW;
11        endcase
12
13    assign out = (state == HIGH);
14 endmodule
```

Choosing the Parameters

Target: 1 kHz at 25% duty, from 48 MHz.

- Cycles per period: _____
- N_H : _____
- N_L : _____
- Counter width: _____

Q: Why `count == limit - 1` and not `count == limit`?

A: _____

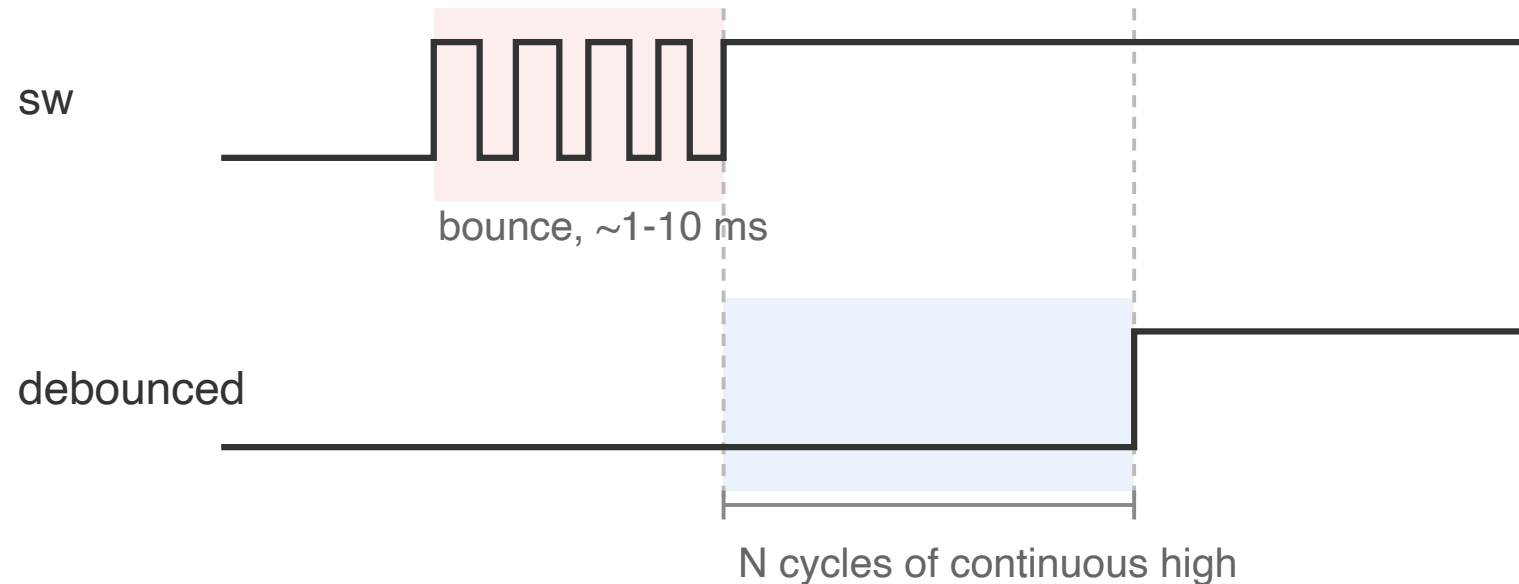
Where You Will Use This

- Dimming an LED
- Motor speed and servo position
- Generating a tone on a speaker
- Lab 5, and very likely your project

Change one parameter and the same circuit does a different job. The controller never changes — only the numbers the counter is told to reach.

FSM Build 1: Switch Debouncer

Why Your Switch Lies To You



- Mechanical contacts do not close cleanly. They chatter for **1–10 ms** before settling.
- On Lab 3's 150 Hz scan clock you mostly get away with it. At the full 48 MHz, 10 ms of bounce is _____ — one press, registered hundreds of times.

Build 1: The Specification

Task: `debounced_sw` goes high only after `sw` has been continuously high for `N` clock cycles. It returns low as soon as `sw` goes low.

A bounce is, by definition, a high that does not *stay* high. So the machine has to be able to
_____ partway through.

We will do this one together, on the board. Build 2 you will do in your groups.

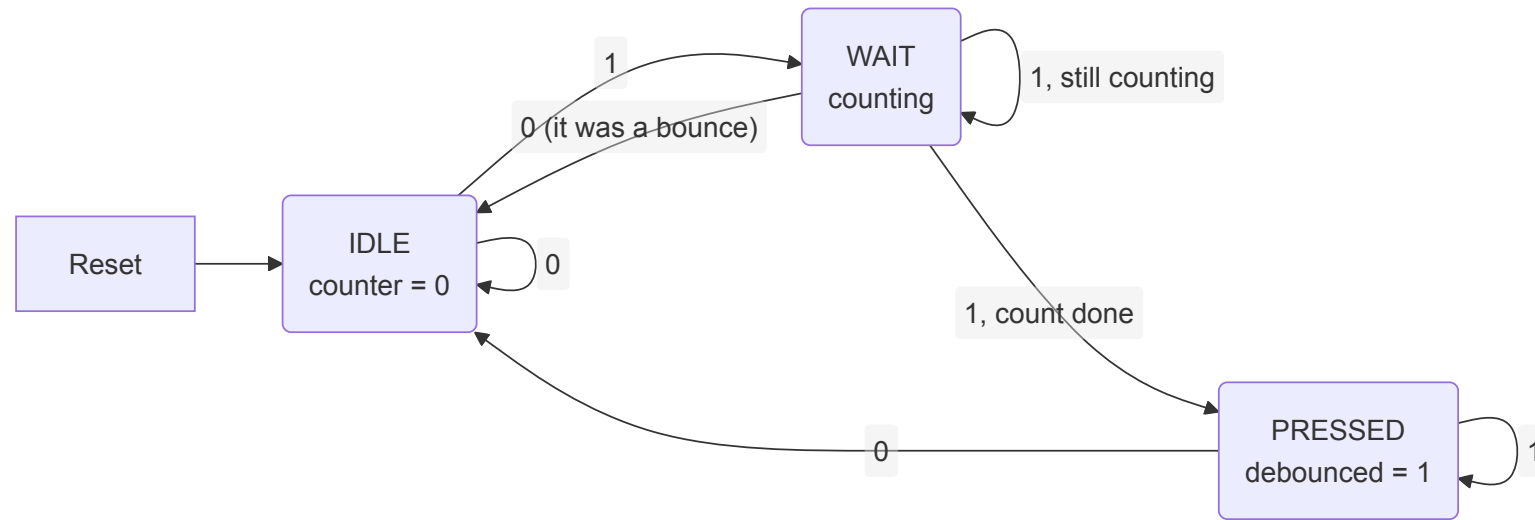
Step 1: List the Specifications

Inputs

Outputs

Internal

Step 2: Sketch the State Transition Diagram



Edge labels are the value of **sw**.

Step 3: A State That Takes Time

Every state we have drawn so far lasted one clock cycle, or sat waiting on an input. **WAIT** is different: it waits on a **counter**, and the coupling runs both directions.

- The FSM **resets** the counter (in **IDLE**)
- The counter **decides** a transition (**WAIT** → **PRESSED**)

Q: Why reset the counter in **IDLE** rather than on the way into **WAIT**?

A:

Steps 4-7: State Register and Counter

```
1  typedef enum logic [1:0] {IDLE, WAIT, PRESSED} statetype;
2  statetype state, nextstate;
3
4  logic [19:0] counter;
5
6  always_ff @(posedge clk, posedge reset)
7      if (reset) state <= IDLE;
8      else      state <= nextstate;
9
10 // The FSM owns the counter: cleared in IDLE, running everywhere else.
11 always_ff @(posedge clk)
12     if (state == IDLE) counter <= 0;
13     else               counter <= counter + 1;
```

Steps 4-7: Next State and Output

```
1  always_comb
2      case (state)
3          IDLE:    nextstate = sw ? WAIT : IDLE;
4          WAIT:    if (!sw)          nextstate = IDLE;      // a bounce
5                   else if (counter[19]) nextstate = PRESSED;
6                   else              nextstate = WAIT;
7          PRESSED: nextstate = sw ? PRESSED : IDLE;
8          default: nextstate = IDLE;
9      endcase
10
11 assign debounced_sw = (state == PRESSED);
```

How Big Should the Counter Be?

At 48 MHz, rejecting 10 ms of bounce needs _____.

Round up to the next power of two: _____

_____.

Q: Why test `counter[19]` instead of `counter == 20'd480000`?

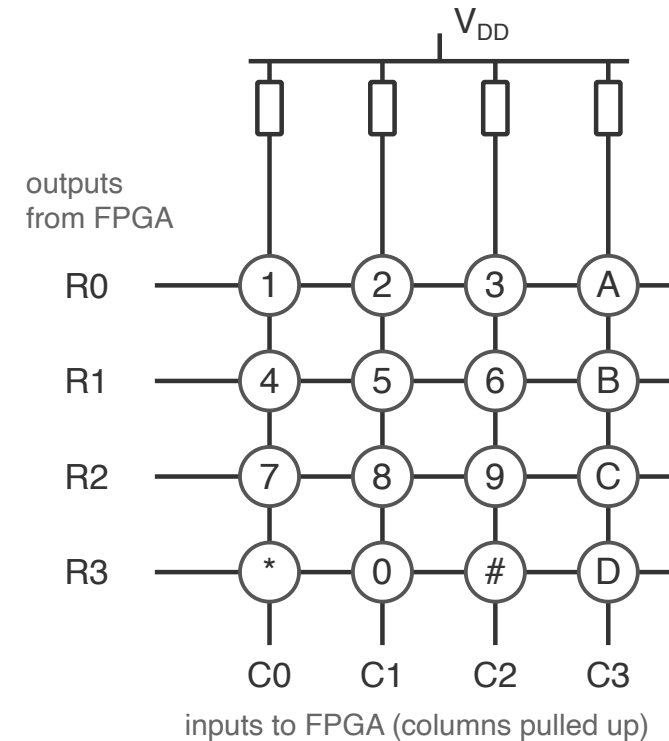
A: _____

FSM Build 2: Keypad Scanner

Lab 3 Preview: Reading a 4x4 Keypad

Lab 3 asks you to read a 16-key keypad and display the last two keys pressed on your seven-segment display.

- 16 keys, but only _____ pins: 4 rows + 4 columns
- Drive **one row low** at a time
- Each column is **pulled up**, so a column reads 1 unless a key in the driven row is pressed
- Pressing the key at (row r , col c) shorts row r to column c



`cols == 4'b1111` therefore means _____

it says nothing about the other three rows.

Why Scan at All?

Q: Why not just give every key its own pin?

A:

Q: We scan the rows at about 150 Hz. Why not run the scanner at the full 48 MHz oscillator speed?

A:

Your Task

Design the FSM that drives the scan.

Given: `clk`, `reset`, and `cols[3:0]` (already synchronized — you wrote that synchronizer last lecture).

Produce: `rows[3:0]` rotating one-hot-low, and `d0/d1` holding the last two keys pressed.

Requirements

1. One key press registers **exactly once**, no matter how long it is held.
2. While a key is held, ignore any other key.
3. All outputs are registered — no glitches on `rows`.

Work in groups of 3. Use the six-step process from Lecture 04.

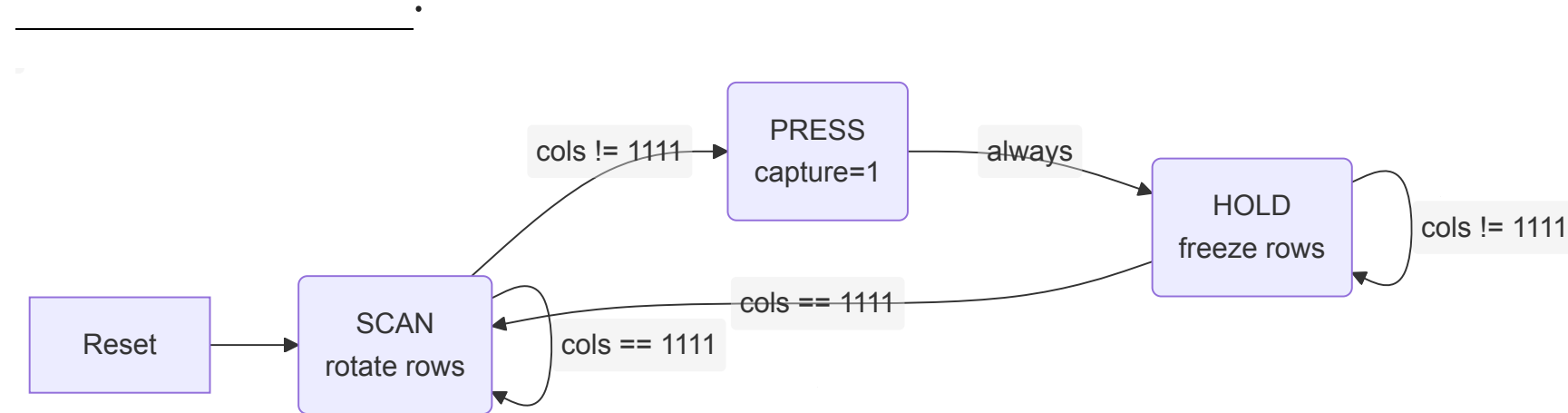
Step 1: List the Specifications

Inputs

Outputs

Step 2: Sketch the State Transition Diagram

Hint: ask yourself what the machine has to *remember*. It is not the key — that lives in a register. It is _____



The Same Three States, Again

	Level-to-pulse (L04)	Debouncer (Build 1)	Keypad scanner (Build 2)
idle	S0 — input low	IDLE — switch up, counter cleared	SCAN — no key, rotate rows
transitional	S1 — emit pulse	WAIT — count out the bounce	PRESS — capture the key
held	S2 — wait for release	PRESSED — hold output high	HOLD — wait for release

Step 3: State Transition Table

Step 4-7: Encode and Implement

```
1  typedef enum logic [2:0] {SCAN = 3'b001, PRESS = 3'b010,  
2                               HOLD = 3'b100} statetype;  
3  statetype state, nextstate;  
4  logic any_key;  
5  
6  assign any_key = ~&cols;      // low-asserted: any column pulled down  
7  
8  always_ff @(posedge clk, posedge reset)  
9      if (reset) state <= SCAN;  
10     else      state <= nextstate;  
11  
12  always_comb  
13      case (state)  
14          SCAN:    nextstate = any_key ? PRESS : SCAN;  
15          PRESS:   nextstate = HOLD;  
16          HOLD:    nextstate = any_key ? HOLD : SCAN;  
17          default: nextstate = SCAN;  
18      endcase
```

Step 4-7: Encode and Implement

The datapath hangs off the state, and all of it is registered:

```
1  always_ff @(posedge clk, posedge reset)
2      if (reset) begin
3          rows <= 4'b0111;
4          d0    <= 4'h0;
5          d1    <= 4'h0;
6      end else begin
7          // Only SCAN moves the row pattern. Everything else freezes it.
8          if (state == SCAN && !any_key) rows <= {rows[0], rows[3:1]};
9
10         if (state == PRESS) begin
11             d1 <= d0;
12             d0 <= key;      // combinational decode of {rows, cols}
13         end
14     end
```

Testing This Is Harder Than It Looks

In hardware `cols` is not an independent input — it **responds** to `rows`. A testbench that wiggles `cols` on a clock schedule will happily produce key combinations that no finger can create.

Model the keypad instead. One key held at (row *R*, col *C*) means:

```
1 // Key '5' sits at row 1, column 1. Holding it ties column 1 to row 1;
2 // every other column stays pulled up.
3 logic key5_down;
4
5 assign cols[1] = key5_down ? rows[1] : 1'b1;
6 assign cols[0] = 1'b1;
7 assign cols[2] = 1'b1;
8 assign cols[3] = 1'b1;
```

Now press and release `key5_down` on a slow schedule and let the DUT scan at its own pace.

How Many Logic Cells Did That Cost?

One logic cell on our FPGA is a 4-input LUT plus a flip-flop, so a registered function of up to 4 inputs is 1 LE.

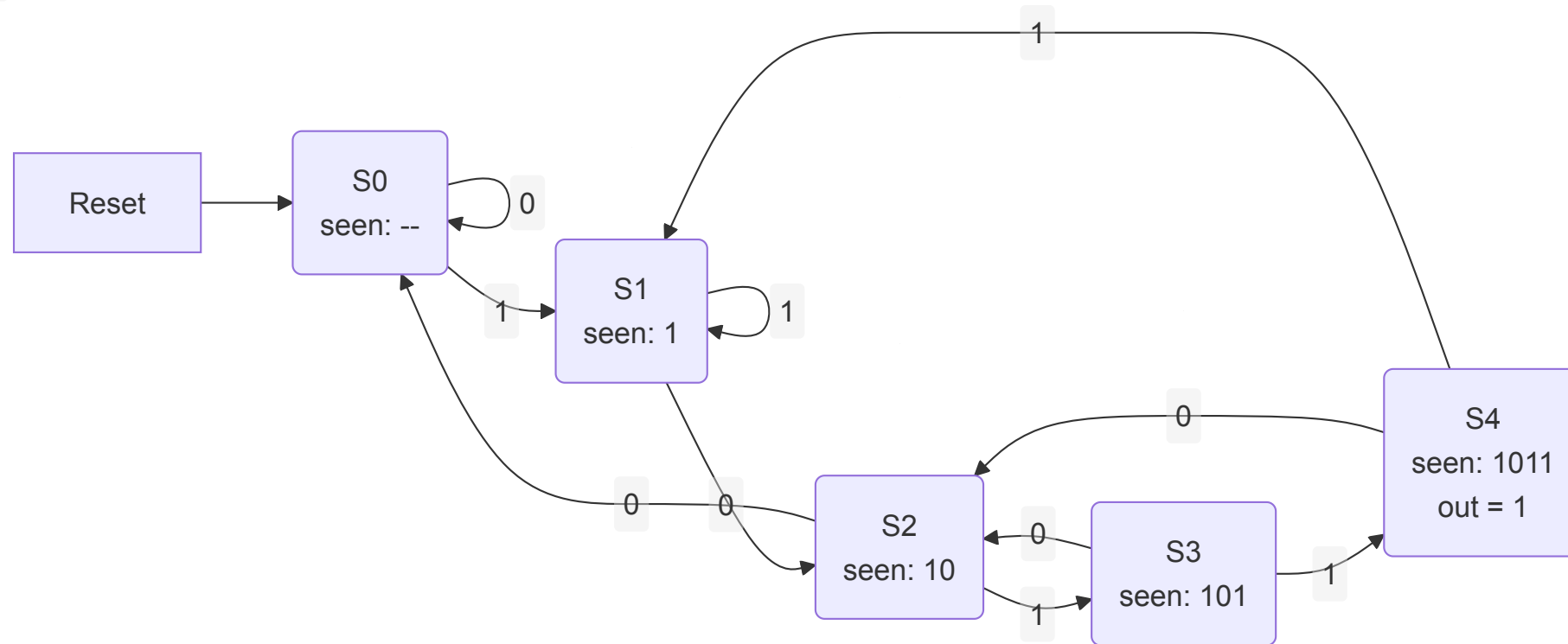
- State register, one-hot, 3 states: _____

- `rows` rotating register: _____
- `d0` and `d1`: _____
- `key` decoder, 4 outputs from `{rows, cols}` (8 inputs): _____

When It Is Not This Skeleton

A Machine That Does Not Fit the Pattern

Detect the bit pattern **1011** in a serial stream, including overlapping matches.



No amount of squinting turns this into idle / transition / held. The machine has to remember _____ — that is genuinely new state, not a rephrasing of “is the input still asserted?”

Wrap up: Metastability

- Metastability cannot be eliminated, only made improbable. MTBF $= T_c e^{(T_c - t_{setup})/\tau} / (NT_0)$ is exponential in how long you wait and linear in everything else, which is why the fix is always one more clock cycle.
- A synchronizer fixes metastability and nothing else. It will not debounce, and it will not keep a multi-bit value coherent.

Wrap up: State Machines

- Canonical, non-canonical, communicating: same hardware underneath, three different ways to describe it. Pick by whether the next state is easier to draw or to compute.
- Control plus datapath is the idea that makes big designs small. A 2-state controller steering a 36,000-state counter is two easy problems instead of one impossible one.
- Three of the four machines we have now looked at share one skeleton: idle, transitional, held. Most FSMs you meet are a machine you have seen before plus a datapath.
- Not all of them, though. When a machine has to remember *how much* of something it has seen, that is real state and no skeleton saves you.
- An FSM that drives a datapath element owns that element: something has to reset the counter, and it should be the state machine.
- When an input responds to your own outputs, your testbench has to model that feedback or it will verify a system that cannot exist.

Announcements/Reminders

- **Lab 3 (keypad scanner)** is posted. *Both* machines we built today are modules in it — start from your state diagrams, not from a blank `.sv` file.
- Reuse your Lab 2 `seven_seg` module and your Lab 4 synchronizer. `cols` is asynchronous and must be synchronized before it reaches the FSM.
- Simulate before you synthesize. Model the keypad in your testbench as shown today.
- Lab 2 checkoffs wrap up this week.